

ECE 385
Spring 2025
Tetris

Final Project

Joshua Joseph (jgj3), Ritvik Patnala (patnala3)
TA: Xuanbo (Bob) Jin
Lab: XJ

Introduction

For our final project in the course we decided to attempt to make a working version of Tetris, with everything the original version had, it is however at a lower resolution since our FPGA boards did not have enough memory to support standard 640x480 resolution. We chose to implement it by imagining the playable grid as a 2D array where we programmed the individual cells to either be colored or empty (0 or 1). It took us the entire time since we finished lab 7.2 to get our game working, we primarily worked on Vivado and only used Vitis in order for our keyboard to work to control the blocks we did not implement audio for the game as we hadn't done it in any of the previous labs so was unsure of the implementation. We are trying to implement a score counter for when each row clears to add a 100 on our FPGA hex displays, we also have working rotating blocks with each space bar press rotating it 90 degrees. We managed to also implement a hard drop and changing colors for the blocks that allows for faster gameplay.

Tetris

Tetris is a tile matching puzzle video game in which the player arranges falling block shapes, known as tetrominoes, into a rectangular playing field. Each tetromino is made up of four connected blocks and comes in one of seven distinct configurations, including a straight line, a four block square, L shapes, and Z shaped patterns. These pieces descend from the top of the screen one at a time, and the player must position and rotate them as they fall to fit them together as efficiently as possible.

The main objective of Tetris is to complete full horizontal lines within the playing field. When a line is completely filled with blocks and contains no empty spaces, it disappears, and all the blocks above it shift down by one row. This action creates more space within the field and allows the game to continue. Players earn points for every line they clear, and clearing multiple lines at once yields higher scores. If the blocks stack up to the top of the playing field and no more pieces can enter, the game ends. Tetris challenges the player's spatial awareness, reaction time, and planning ability.

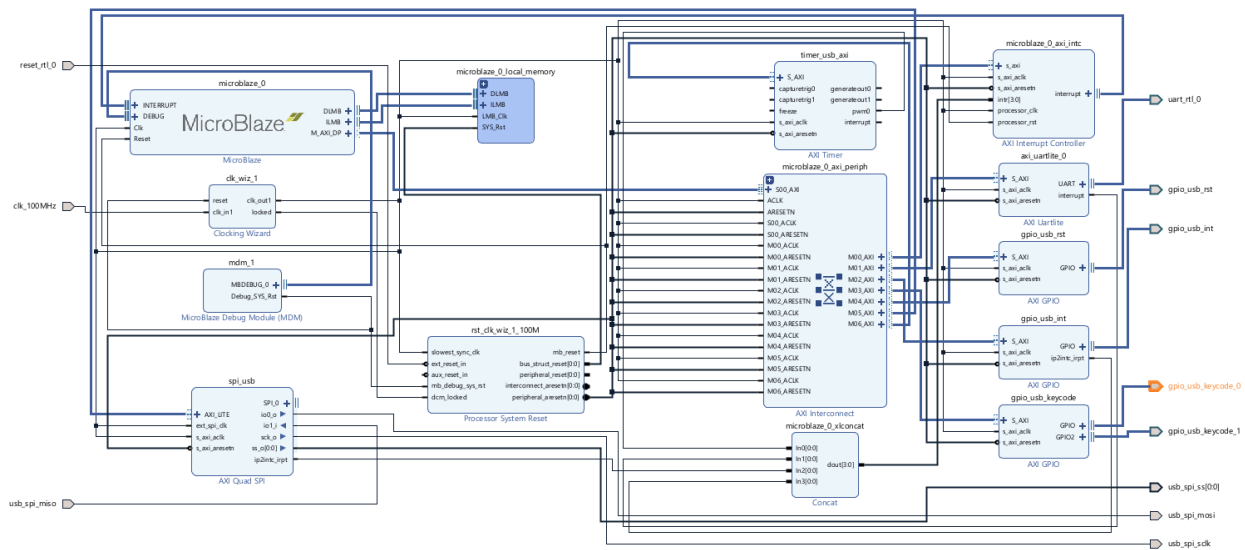
Background

For the background we used one of the CAs tools that helped them when they took the class to make the background show up on the monitor. The attached image shows exactly how we implemented our tetris background.

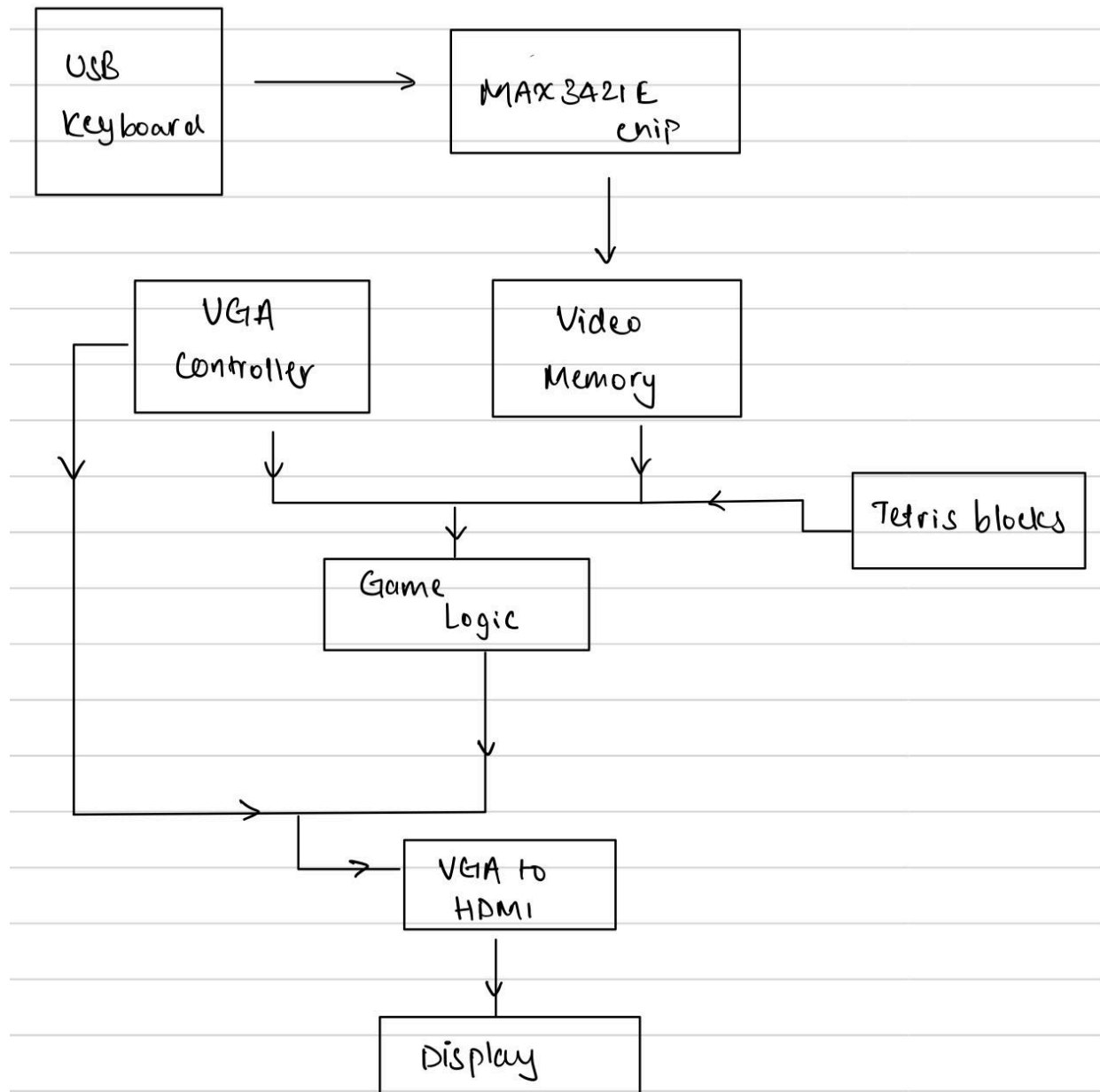
How to use:

1. Download Python 3 (developed on 3.11.1 for Windows 10).
2. Download this entire repository.
3. Open a terminal inside this repository. (PowerShell on Windows) (Optional: Use a python virtual environment (Not Recommended), instructions below).
4. Run `pip install -r requirements.txt`. This installs all the required packages.
5. Run `python main.py`.
6. Follow the instructions in the terminal. There are two provided images to try out: `cat.jpg` and `butterfly.jpg`.
7. Look at the output image in the generated folder and verify that your chosen settings look okay.
8. In the IP Catalog look up "Block Memory Generator" under Memories and Storage Elements.
9. Change the Component name to `<image_name>_rom`, Memory Type to Single Port Rom and Port A Width and Depth according to what the Image has generated. Disable the Enable Port Type (Set it as Always enabled).
10. Add the generated .COE file in the Other Options page. You do not need to fill the remaining Memory Locations.
11. Load the .sv files (palette, and example) to your Vivado project.
12. Instantiate the example module in your project, connect all the signals, compile, program, and verify that you see your generated image on the screen!

Block Diagram



Design Block Diagram



.SV Files

Module: VGA_controller.sv

Inputs: pixel_clk, reset

Outputs: hs, vs, active_nblank, sync, drawX [9:0], drawY [9:0]

Purpose: This module generates the necessary timing and control signals to interface digital video output with a VGA-compatible display.

Description: The vga_controller module is a SystemVerilog implementation designed to generate standard VGA timing signals for a 640x480 resolution display using a 25 MHz pixel clock. It includes horizontal and vertical counters to track pixel positions, generates horizontal and vertical sync pulses according to VGA specifications, and asserts an active_nblank signal to indicate when the current pixel is within the visible region of the screen. The module outputs current pixel coordinates drawX and drawY and disables composite sync for simplified usage.

Module: tetris_background_example.sv

Inputs: DrawX [9:0], DrawY [9:0], blank, vga_clk

Outputs: Red [3:0], Green [3:0], Blue [3:0]

Purpose: The purpose of this module is to provide a scalable, color-mapped background for VGA-based graphics applications, particularly for games like Tetris. It separates visual data (ROM-stored image indices) from color definitions, making it efficient to update the visual theme by simply modifying ROM or palette content. This module is ideal for embedded graphics projects where resource-efficient image rendering and modular design are important.

Description: The tetris_background_example module generates a full-screen background image for a Tetris-style video display by sampling pixel data from a ROM and converting it into RGB color values. It calculates a ROM address based on the current screen coordinates scaled to a 320x240 background resolution stretched to fit a 640x480 VGA output. On every negative edge of the VGA clock, a background pixel value is read from ROM; then, on the positive edge, if the video signal is not blanked, this value is mapped to corresponding RGB color values via a palette module and output to the screen. This mechanism enables smooth and visually appealing background rendering synchronized with the VGA display pipeline.

Module: hdmi_text_controller_v1_0.sv

Inputs: axi_aclk, axi_aresetn, axi_awaddr, axi_awprot [2:0], axi_awvalid, axi_wdata, axi_wstrb, axi_wvalid, axi_bready, axi_araddr, axi_arprot [2:0], axi_arvalid, axi_rready

Outputs: hdmi_clk_n, hdmi_clk_p, hdmi_tx_n [2:0], hdmi_tx_p [2:0], axi_awready, axi_wready, axi_bresp, axi_bvalid, axi_arready, axi_rdata, axi_rresp, axi_rvalid

Purpose: This module serves as a complete hardware controller for displaying colored text over HDMI. It provides programmable video memory access via an AXI interface, allowing a user or processor to manipulate text content in real time. The architecture separates functional concerns—synchronization, font fetching, and color generation—to ensure modularity and scalability.

Description: The hdmi_text_controller_v1_0 is a top-level SystemVerilog module designed to drive HDMI output for a text-based display system using AXI4 bus communication. It integrates several functional blocks including a clocking wizard, VGA synchronization generator (vga_controller), AXI interface handler (hdmi_text_controller_v1_0_AXI), font address calculator, font ROM access, and color mapping. Finally, the module routes processed RGB and sync signals through an HDMI transmitter that outputs TMDS signals to an HDMI display. The

design supports drawing characters using font ROM and maps their visual representation onto an HDMI-compatible signal stream.

Module: hdmi_text_controller_v1_0_AXI.sv

Inputs: drawX [9:0], drawY [9:0], S_AXI_ACLK, S_AXI_ARESETN, S_AXI_AWADDR, S_AXI_AWPROT [2:0], S_AXI_AWVALID, S_AXI_WDATA, S_AXI_WSTRB, S_AXI_WVALID, S_AXI_BREADY, S_AXI_ARADDR, S_AXI_ARPROT [2:0], S_AXI_ARVALID, S_AXI_RREADY

Outputs: control_reg_data [31:0], slv_reg_data [31:0], S_AXI_AWREADY, S_AXI_WREADY, S_AXI_BRESP [1:0], S_AXI_BVALID, S_AXI_ARREADY, S_AXI_RDATA, S_AXI_RRESP, S_AXI_RVALID

Purpose: Provides AXI4-Lite interface logic for accessing and modifying internal memory-mapped registers. Maps 2D screen coordinates into a flat register index to retrieve character data for display. Serves as a bridge between software and hardware-rendered HDMI/VGA output. Stores configuration or control data at a designated register which can be read externally through control_reg_data. Acts as a hardware peripheral for character-based rendering in text display systems like simple video terminals or graphics labs.

Description: The hdmi_text_controller_v1_0_AXI module implements an AXI4-Lite slave interface to manage and control an HDMI text rendering system. This SystemVerilog design handles standard AXI transactions, including read and write operations, and maps them to an array of internal registers (slv_regs). These registers can store character or control data for a virtual text display memory. The module supports indexed memory access, enabling users to update or retrieve visual data based on pixel coordinates and provides access to both control information and current register values.

Module: hex_driver.sv

Inputs: clk, reset, [4:0] in[4]

Outputs: hex_seg [7:0], hex_grid [3:0]

Description: Our hex module is responsible for displaying four 4-bit binary values on a multiplexed 7-segment display. It first converts each 4-bit nibble into its corresponding 7-segment representation using the nibble_to_hex instantiation inside the module. It has stored hex values for the number of bits it is to receive using a 17-bit counter.

Purpose: Our hex module drives a 7-segment display by converting four 4-bit inputs into hexadecimal representations and multiplexing the output to display each digit in sequence.

Module: game_over_example.sv

Inputs: DrawX [9:0], DrawY [9:0], blank, vga_clk

Outputs: Red [3:0], Green [3:0], Blue [3:0]

Purpose: The purpose of this module is to render a visual "Game Over" screen in VGA-based gaming applications. It provides a simple and modular method for displaying pre-rendered

full-screen graphics using ROM and a color palette, enabling efficient resource usage and ease of updating graphics without changing the logic structure.

Description: The game_over_example module displays a full-screen "Game Over" image on a VGA display by mapping screen coordinates to a ROM-based pixel index and converting it to RGB color through a palette. It scales a 320×240 pixel image to fill a 640×480 VGA resolution, using calculated ROM addresses derived from the current pixel's DrawX and DrawY values. The ROM is accessed on the negative edge of the VGA clock to fetch a 3-bit color index, which is translated into 4-bit red, green, and blue values using a palette module. On the positive clock edge, these RGB values are driven to the screen only when the video is active.

Module: mb_usb_hdmi_top.sv

Inputs: Clk, reset_rtl_0, gpio_usb_int_tri_i [0:0], usb_spi_miso, uart_rtl_0_rxd

Outputs: gpio_usb_rst_tri_o, usb_spi_mosi, usb_spi_sclk, usb_spi_ss, uart_rtl_0_txd, hdmi_tmds_clk_n, hdmi_tmds_clk_p, hdmi_tmds_data_n [2:0], hdmi_tmds_data_p [2:0], hex_segA [7:0], hex_gridA [3:0], hex_segB [7:0], hex_gridB [3:0]

Purpose: Top-level integration module combining USB host, UART, and HDMI output with a Tetris game interface and score display using MicroBlaze.

Description: The mb_usb_hdmi_top module serves as the top-level integration point for a system that interfaces with USB, UART, and HDMI peripherals while managing a simple Tetris-style game display. It connects a MicroBlaze-based block (mb_block) with USB host control, UART communication, and SPI-based USB interfacing. The design also incorporates an HDMI transmitter by converting VGA signals into TMDS format using the hdmi_tx_0 module. Internally, the module coordinates clock signals via a clock wizard, generates VGA timing signals, and drives display content using tetris_background_example, game_over_example, and tetris_grid. It also decodes the game score into human-readable format using a seven-segment display driver. This module manages synchronization across different clock domains and ensures reset logic is properly aligned with the expectations of submodules.

Module: tetris_grid.sv

Inputs: drawX [9:0], drawY [9:0], frame_clk, reset, keycode [7:0], bg_red [3:0], bg_green [3:0], bg_blue [3:0], end_red [3:0], end_green [3:0], end_blue [3:0]

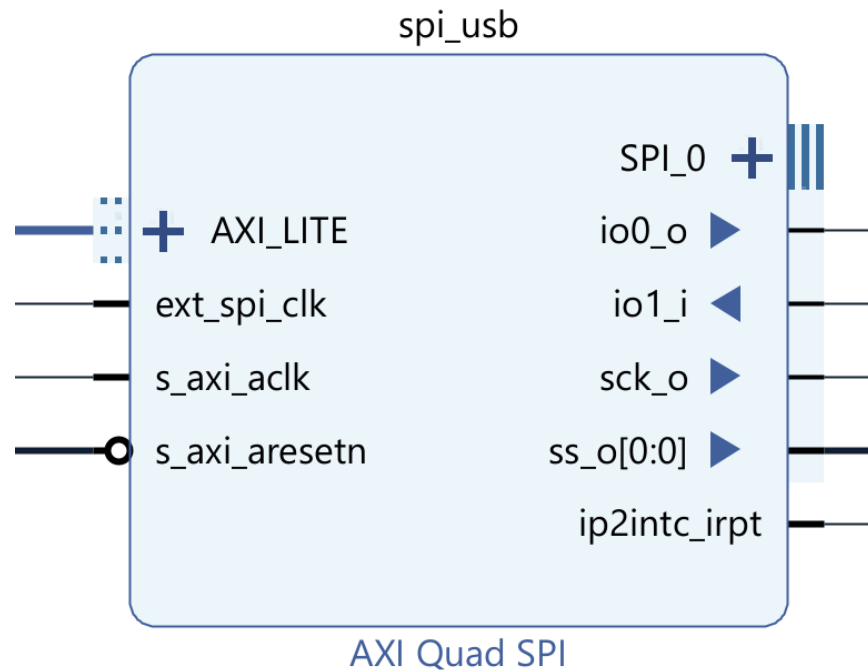
Outputs: red [3:0], green [3:0], blue [3:0], score [15:0]

Purpose: Implements the main Tetris game logic, rendering RGB output and updating score based on keyboard input and frame timing.

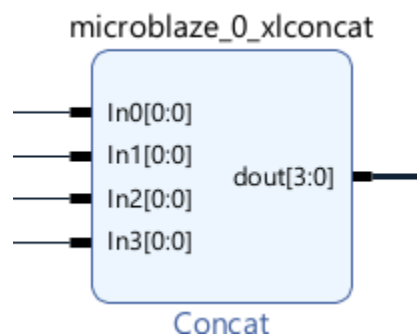
Description: The tetris_grid module is responsible for managing the core logic of the Tetris game, generating the RGB pixel data for each frame based on the current game state. It interprets USB keyboard input from the MicroBlaze system, updates the position and status of Tetris blocks on a virtual grid, and handles the mechanics of piece movement, rotation, line clearing, and score accumulation. On each frame synchronized with vertical sync, it overlays game content onto a colored background and conditionally displays a game over screen by blending

background and endgame color signals. The module dynamically outputs red, green, and blue color values based on the location of each pixel, and updates a 16-bit score signal to be shown on external displays.

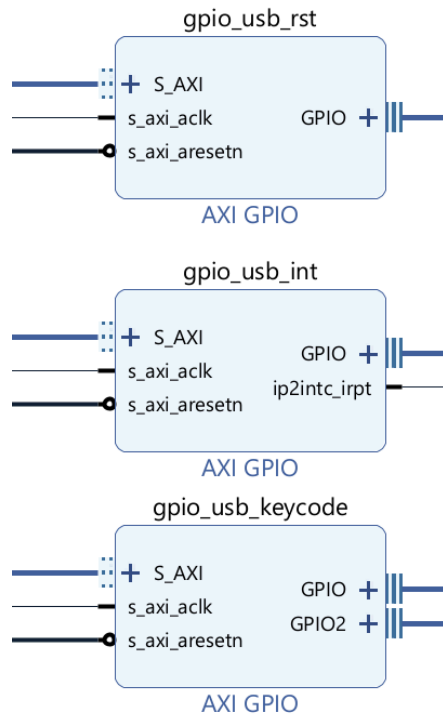
Block Components:



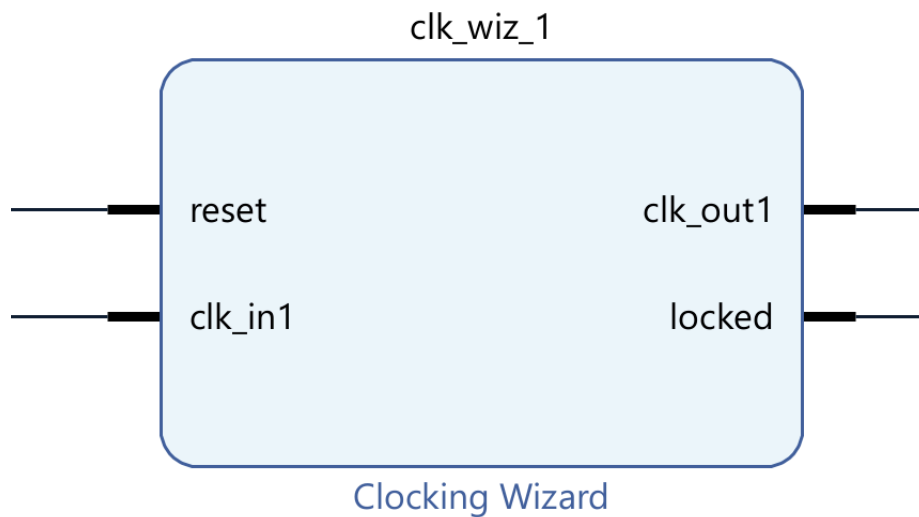
Description: This module is what allows for communication between our Microblaze and the SPI peripherals, this communication allows for the MAX3421E chip to interact with the Microblaze using the AXI bus.



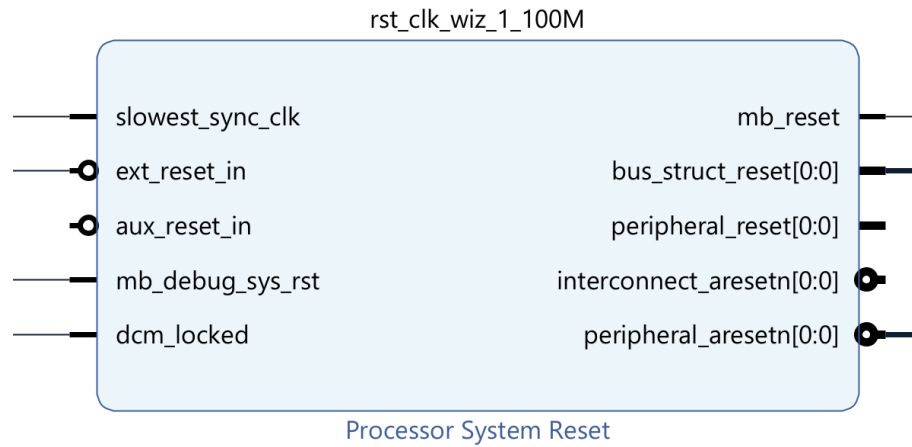
Description: Its job is to combine signals together and output these signals as a single concatenated signal, for us we have it combining all the interrupts to then enter into our interrupt controller after this.



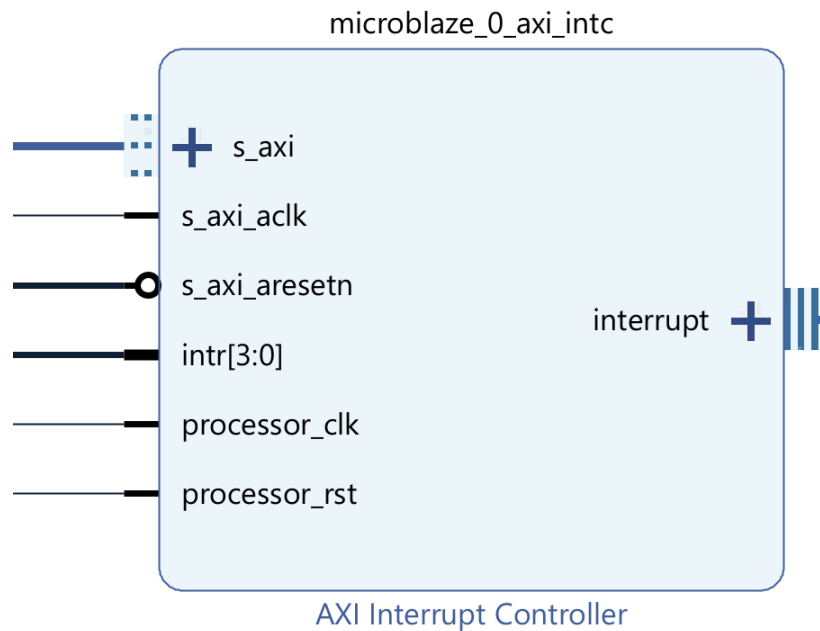
Description: These blocks are used as basic AXI general input and output modules that can be connected to any device in order to interface with the AXI bus, which allows for the Microblaze to interact with our FPGA.



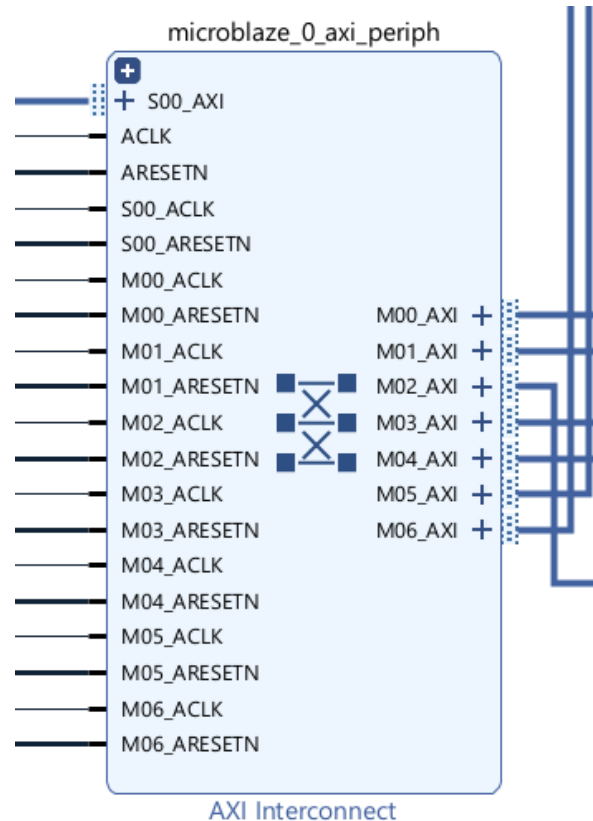
Description: The clocking wizard is a tool that takes the input clock and convert that clock to the clock you actually want and that allows us to generate a clock of a desired frequency that keeps the entire system synchronized.



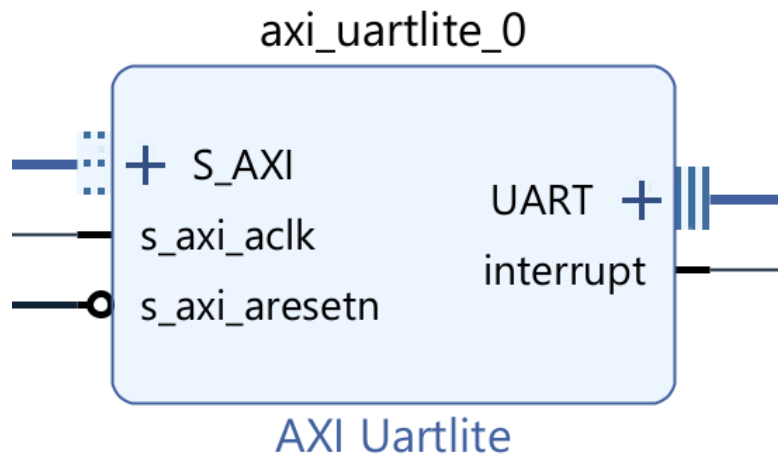
Description: It can reset the processor system along with its peripherals this allows for easy starting up state when reset is performed.



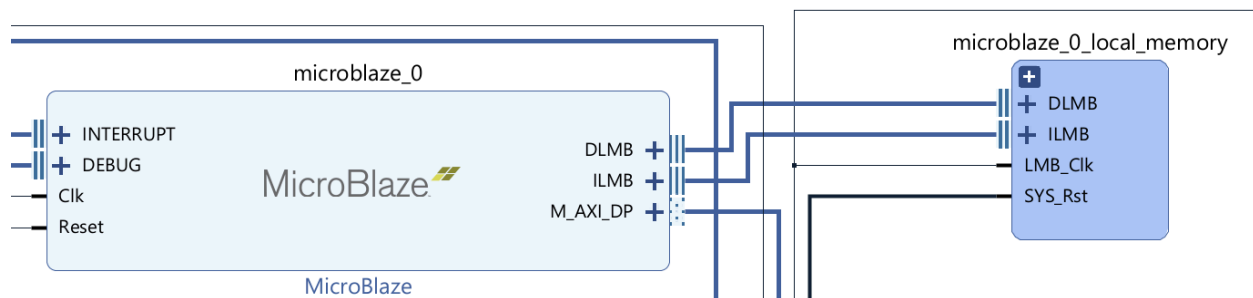
Description: It is the main mechanism that controls the interrupts using the AXI standard, which are generated either by our peripherals or our FPGA. Overall all the interrupts will depend on this module.



Description: In order to connect all components in the AXI bus together we need a module where they all converge. This block is where they converge to allow for efficient communication between all the components involved in the AXI bus.



Description: An AXI-based receiver that acts synchronously between the receiver and transmitter module. We have connections in this module that can interface with the AXI bus and we use it to communicate with the Vitis program which allows microblaze to interface with software.



Description: The Microblaze processor on its own doesn't really have any intended use on its own however once connected to the rest of the IP modules using AXI4 module to connect the bus to the Microblaze. Essentially the Microblaze is a soft core processor and functions as a customizable embedded processor which is only usable after adding multiple peripherals to the block diagram. It is configurable based on our needs, acts as a black box processor that writes C code and can also access the AXI4 bus. It is a lower performing processor which is a 32 bit modified Harvard RISC architecture type, it can have multipliers or floating point units which increases the speed when it comes to the structures that are occupying space. Typically Harvard structures have a separate bus but in our case our microcontroller is able to also access the bus as well.

Software

We needed to use software so that we could use the keyboard to have the movements and the rotating button.

SPI (Serial Peripheral Interface) Protocol:

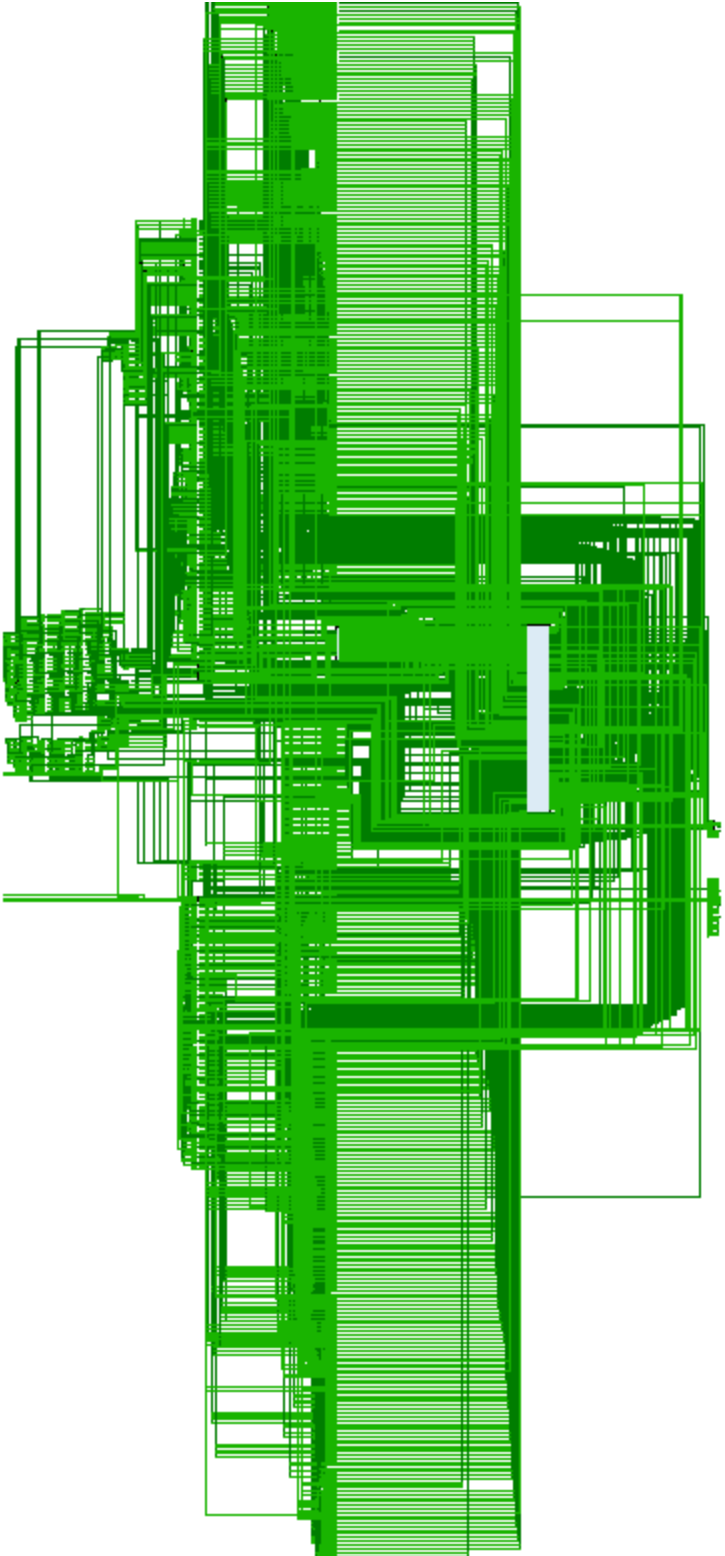
The SPI protocol is a synchronous, full-duplex serial communication standard used to transfer data between a master device, our microblaze and one or more slave peripherals, like the MAX3421E USB controller. It consists of four signals: SCLK, MOSI, MISO and SS.

Communication is coordinated by the master, which drives the SCLK and SS lines and controls the data flow. In the context of the MAX3421E, all register accesses both reads and writes are performed using SPI. A typical SPI transaction begins when the master asserts the SS line low to select the MAX3421E. The first byte sent over the MOSI line is the command byte, which encodes the target register address and the direction of the transaction using a read/write bit. For example, setting the inverted read and write bits to 0 indicates a write operation, while setting it to 1 indicates a read. After the command byte, one or more data bytes follow, depending on the nature of the transaction. In a write operation, the data bytes are transmitted from the master to the MAX3421E. In a read operation, the master sends out empty bytes, and the MAX3421E responds with the requested data bytes over the MISO line. This dummy exchange is necessary because SPI is full-duplex: data is sent and received simultaneously on each clock cycle.

Although the MAX3421E always returns a status byte during the command byte phase, it is typically ignored in host mode. Additionally, when the MAX3421E is receiving data it returns zeros on MISO; this behavior is expected and is not an error. As an example, writing the byte

0x01 to register 0x1E involves sending a command byte of 0xF2 followed by the data byte 0x01. The SPI peripheral in hardware handles the low-level bit manipulation and clocking, allowing the programmer to work at the byte level for simplicity.

Top Level Schematic



Design Statistics

LUT	17052
DSP	6
Memory (BRAM)	58.5
Flip-Flop	3591
Frequency	WNS = -1.243 114.18131993606 MHz
Static Power	0.078
Dynamic Power	0.411
Total Power	0.489

Conclusion

Overall, we enjoyed the game making but it did take a lot of dedication and we had some serious problems each time when we were trying to change the colours for individual blocks, this made us have to remake the project three different times which was very time consuming. On top of that our bitstreams would take 7 minutes to complete so it made us slower towards the end. The blocks were probably the easiest thing to design after we moved away from the sprites approach and instead used a grid. This allowed us to draw easily which got us in a better headspace but after having to restart three different times prevented us from getting a score on our screen. In the end we are very happy with our project and hopefully it can get us that A grade in the course as we really dedicated hours to this course each week, ever since our Lab 4 extension we worked on 385 6 days a week.